

2. Write the python program to solve 8-Queen problem

Aim

To develop a Python program to solve the **8-Queen Problem** using the **Backtracking Algorithm** by accepting the **board size (N)** from the user and placing **N queens** on an **N × N chessboard** such that no two queens attack each other.

Algorithm

1. Start the program.
2. Read the number of queens (N) from the user.
3. Create an empty $N \times N$ chessboard.
4. Begin placing queens column by column.
5. For each row in the current column:
 - Check whether placing the queen is safe.
 - If safe, place the queen.
 - Recursively place queens in the next column.
 - If a solution is found, stop.
 - Otherwise, remove the queen (Backtrack).
6. Repeat until all queens are placed.
7. Display the chessboard if a solution exists; otherwise display "No Solution Exists."
8. Stop.

Program

```
# Function to check whether a queen can be placed safely
def is_safe(board, row, col, n):

    # Check left side of current row
    for i in range(col):
        if board[row][i] == 1:
            return False

    # Check upper-left diagonal
    i = row
    j = col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    # Check lower-left diagonal
    i = row
    j = col
    while i < n and j >= 0:
        if board[i][j] == 1:
            return False
        i += 1
        j -= 1
```

```

        j -= 1

    return True

# Backtracking function
def solve_nqueen(board, col, n):

    if col >= n:
        return True

    for i in range(n):
        if is_safe(board, i, col, n):

            board[i][col] = 1

            if solve_nqueen(board, col + 1, n):
                return True

            board[i][col] = 0

    return False

# Main Program
n = int(input("Enter the number of Queens (N): "))

board = [[0 for i in range(n)] for j in range(n)]

if solve_nqueen(board, 0, n):

    print("\nSolution Exists:\n")

    for row in board:
        for cell in row:
            if cell == 1:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()

else:
    print("No Solution Exists.")

```

Input

Enter the number of Queens (N): 8

Output

Solution Exists:

```
Q . . . . . . . .  
. . . . . Q .  
. . . . Q . . .  
. . . . . . Q  
. Q . . . . . .  
. . . Q . . . .  
. . . . Q . . .  
. . Q . . . . .  
>>> |
```

Result

The Python program was successfully implemented to solve the **8-Queen Problem** using the **Backtracking Algorithm**. The program accepts the **number of queens (N)** from the user, places the queens on an **$N \times N$ chessboard** such that no two queens attack each other, and displays one valid solution if it exists. This demonstrates the effectiveness of the backtracking technique in solving constraint satisfaction problems.